

## 12.5 Lab: Unsupervised Learning

Trong lab này, ta minh họa PCA và clustering trên một số datasets. Như ở các lab khác, ta import một số libraries ngay từ đầu. Cách này làm code dễ đọc hơn, vì chỉ cần nhìn vài dòng đầu của notebook là có thể biết những libraries nào được sử dụng.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.datasets import get_rdataset
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from ISLP import load_data
```

Ta cũng tập hợp các imports mới cần cho lab này.

```
In [2]: from sklearn.cluster import \
        (KMeans,
         AgglomerativeClustering)
from scipy.cluster.hierarchy import \
        (dendrogram,
         cut_tree)
from ISLP.cluster import compute_linkage
```

### 12.5.1 Principal Components Analysis

Trong lab này, ta thực hiện PCA trên **USArrests**, một data set trong R computing environment. Ta lấy data bằng `get_rdataset()`, hàm có thể fetch data từ nhiều standard R packages.

`get_rdataset()`

Các rows của data set chứa 50 states, theo thứ tự alphabet.

```
In [3]: USArrests = get_rdataset('USArrests').data
USArrests
```

```
Out[3]:
```

|           | Murder | Assault | UrbanPop | Rape |
|-----------|--------|---------|----------|------|
| Alabama   | 13.2   | 236     | 58       | 21.2 |
| Alaska    | 10.0   | 263     | 48       | 44.5 |
| Arizona   | 8.1    | 294     | 80       | 31.0 |
| ...       | ...    | ...     | ...      | ...  |
| Wisconsin | 2.6    | 53      | 66       | 10.8 |
| Wyoming   | 6.8    | 161     | 60       | 15.6 |

Các columns của data set chứa bốn variables.

```
In [4]: USArrests.columns
```

```
Out[4]: Index(['Murder', 'Assault', 'UrbanPop', 'Rape'], dtype='object')
```

Trước hết ta examine data một cách ngắn gọn. Ta thấy các variables có means rất khác nhau.

```
In [5]: USArrests.mean()
```

```
Out[5]: Murder      7.788
Assault    170.760
UrbanPop   65.540
Rape       21.232
dtype: float64
```

Dataframes có một số useful methods để compute column-wise summaries. Ta cũng có thể examine variance của bốn variables bằng method `var()`.

```
In [6]: USArrests.var()
```

```
Out[6]: Murder      18.970465
Assault    6945.165714
UrbanPop   209.518776
Rape       87.729159
dtype: float64
```

Không ngạc nhiên khi các variables cũng có variances rất khác nhau. Variable **UrbanPop** đo percentage của population trong mỗi state sống ở urban area, nên không thể so trực tiếp với number of rapes trên 100,000 individuals. PCA tìm các derived variables giải thích phần lớn variance trong data set. Nếu ta không scale variables trước khi thực hiện PCA, principal components sẽ chủ yếu bị chi phối bởi **Assault**, vì variable này có variance lớn hơn hẳn. Vì vậy, nếu variables được đo bằng different units hoặc khác nhau nhiều về scale, nên standardize để chúng có standard deviation bằng one trước khi thực hiện PCA. Thông thường ta cũng set means bằng zero.

Việc scaling này có thể thực hiện bằng transform `StandardScaler()` đã import ở trên. Trước hết ta fit scaler, qua đó compute các means và standard deviations cần thiết, rồi apply nó lên data bằng method `transform`. Như trước, ta combine hai bước bằng `fit_transform()`.

```
In [7]: scaler = StandardScaler(with_std=True,
                                with_mean=True)
USArrests_scaled = scaler.fit_transform(USArrests)
```

*PCA()* Sau khi scale data, ta có thể thực hiện principal components analysis bằng transform `PCA()` từ package `sklearn.decomposition`.

```
In [8]: pcaUS = PCA()
```

Theo default, `PCA()` centers variables để có mean zero nhưng không scale chúng. Transform `pcaUS` có thể dùng để tìm PCA scores trả về sau khi fit. Khi method `fit()` đã được gọi, object `pcaUS` cũng chứa một số quantities hữu ích.

```
In [9]: pcaUS.fit(USArrests_scaled)
```

Sau khi fitting, attribute `mean_` tương ứng với means của variables. Trong trường hợp này, vì ta đã center và scale data bằng `scaler`, các means đều bằng 0.

```
In [10]: pcaUS.mean_
```

```
Out[10]: array([-0.,  0., -0.,  0.])
```

Scores có thể được compute bằng method `transform()` của `pcaUS` sau khi object đã được fit.

```
In [11]: scores = pcaUS.transform(USArrests_scaled)
```

Ta sẽ plot các scores này ở phía dưới. Attribute `components_` cung cấp principal component loadings: mỗi row của `pcaUS.components_` chứa corresponding principal component loading vector.

```
In [12]: pcaUS.components_
```

```
Out[12]: array([[ 0.53589947,  0.58318363,  0.27819087,  0.54343209],
 [ 0.41818087,  0.18798560, -0.87280619, -0.16731864],
 [-0.34123273, -0.26814843, -0.37801579,  0.81777791],
 [ 0.64922780, -0.74340748,  0.13387773,  0.08902432]])
```

Biplot là một visualization method thường dùng cùng PCA. Nó không được tích hợp như một standard part của `sklearn`, dù có các Python packages tạo loại plot này. Ở đây ta tự tạo một simple biplot.

```
In [13]: i, j = 0, 1 # which components
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(scores[:,0], scores[:,1])
ax.set_xlabel('PC%d' % (i+1))
ax.set_ylabel('PC%d' % (j+1))
for k in range(pcaUS.components_.shape[1]):
    ax.arrow(0, 0,
             pcaUS.components_[i,k],
             pcaUS.components_[j,k])
    ax.text(pcaUS.components_[i,k],
            pcaUS.components_[j,k],
            USArrests.columns[k])
```

Figure này là reflection của Figure 12.1 qua y-axis. Nhớ rằng principal components chỉ unique đến một sign change, nên ta có thể reproduce figure đó bằng cách flip signs của second set of scores và loadings. Ta cũng increase length của arrows để nhấn mạnh loadings.

```
In [14]: scale_arrow = s_ = 2
scores[:,1] *= -1
pcaUS.components_[1] *= -1 # flip the y-axis
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(scores[:,0], scores[:,1])
ax.set_xlabel('PC%d' % (i+1))
ax.set_ylabel('PC%d' % (j+1))
for k in range(pcaUS.components_.shape[1]):
    ax.arrow(0, 0,
             s_*pcaUS.components_[i,k],
             s_*pcaUS.components_[j,k])
    ax.text(s_*pcaUS.components_[i,k],
            s_*pcaUS.components_[j,k],
            USArrests.columns[k])
```

Standard deviations của principal component scores như sau:

```
In [15]: scores.std(0, ddof=1)
```

```
Out[15]: array([1.5909, 1.0050, 0.6032, 0.4207])
```

Variance của mỗi score có thể được extract trực tiếp từ object `pcaUS` qua attribute `explained_variance_`.

```
In [16]: pcaUS.explained_variance_
```

```
Out[16]: array([2.5309, 1.01, 0.3638, 0.177])
```

Proportion of variance explained bởi mỗi principal component (PVE) được lưu trong `explained_variance_ratio_`:

```
In [17]: pcaUS.explained_variance_ratio_
```

```
Out[17]: array([0.6201, 0.2474, 0.0891, 0.0434])
```

Ta thấy first principal component giải thích 62.0% variance trong data, next principal component giải thích 24.7%, và cứ thế. Ta có thể plot PVE explained bởi mỗi component cũng như cumulative PVE. Trước hết plot proportion of variance explained.

```
In [18]: %%capture
fig, axes = plt.subplots(1, 2, figsize=(15, 6))
ticks = np.arange(pcaUS.n_components_)+1
ax = axes[0]
ax.plot(ticks,
        pcaUS.explained_variance_ratio_,
        marker='o')
ax.set_xlabel('Principal Component')
ax.set_ylabel('Proportion of Variance Explained')
ax.set_ylim([0,1])
ax.set_xticks(ticks)
```

Lưu ý việc dùng `%%capture`, giúp suppress việc display một figure đang mới được build một phần.

```
In [19]: ax = axes[1]
ax.plot(ticks,
        pcaUS.explained_variance_ratio_.cumsum(),
        marker='o')
ax.set_xlabel('Principal Component')
ax.set_ylabel('Cumulative Proportion of Variance Explained')
ax.set_ylim([0, 1])
ax.set_xticks(ticks)
fig
```

Result tương tự Figure 12.3. Method `cumsum()` computes cumulative sum của các elements trong một numeric vector. Chẳng hạn:

*cumsum()*

```
In [20]: a = np.array([1,2,8,-3])
np.cumsum(a)
```

```
Out[20]: array([ 1,  3, 11,  8])
```

### 12.5.2 Matrix Completion

Bây giờ ta re-create analysis đã thực hiện trên `USArrests` data trong Section 12.3. Ta đã thấy ở Section 12.2.2 rằng solving optimization problem (12.6) trên một centered data matrix  $X$  tương đương với computing first  $M$  principal components của data. Ta dùng scaled và centered `USArrests` data làm  $X$  bên dưới. Singular value decomposition (SVD) là một general algorithm để solve (12.6).

*singular value  
decomposition;  
svd()*

```
In [21]: X = USArrests_scaled
         U, D, V = np.linalg.svd(X, full_matrices=False)
         U.shape, D.shape, V.shape
```

```
Out[21]: ((50, 4), (4,), (4, 4))
```

Function `np.linalg.svd()` returns three components,  $U$ ,  $D$  và  $V$ . Matrix  $V$  tương đương loading matrix từ principal components, ngoại trừ một unimportant sign flip. Option `full_matrices=False` đảm bảo rằng với một tall matrix, shape của  $U$  giống shape của  $X$ .

```
In [22]: V
```

```
Out[22]: array([[ -0.53589947, -0.58318363, -0.27819087, -0.54343209],
                [  0.41818087,  0.18798560, -0.87280619, -0.16731864],
                [-0.34123273, -0.26814843, -0.37801579,  0.81777791],
                [  0.64922780, -0.74340748,  0.13387773,  0.08902432]])
```

```
In [23]:
```

```
pcaUS.components_
```

```
Out[23]: array([[ 0.53589947,  0.58318363,  0.27819087,  0.54343209],
                [  0.41818087,  0.18798560, -0.87280619, -0.16731864],
                [-0.34123273, -0.26814843, -0.37801579,  0.81777791],
                [  0.64922780, -0.74340748,  0.13387773,  0.08902432]])
```

Matrix  $U$  tương ứng với một standardized version của PCA score matrix, trong đó mỗi column được standardized để có sum-of-squares bằng one. Nếu multiply mỗi column của  $U$  với corresponding element của  $D$ , ta recover PCA scores exactly, ngoại trừ một meaningless sign flip.

```
In [24]:
```

```
(U * D[None,:])[:3]
```

```
Out[24]: array([[ -0.9856,  1.1334, -0.4443,  0.1563],
                [-1.9501,  1.0732,  2.0400, -0.4386],
                [-1.7632, -0.7460,  0.0548, -0.8347]])
```

```
In [25]:
```

```
scores[:3]
```

```
Out[25]: array([[ 0.9856, -1.1334, -0.4443,  0.1563],
                [ 1.9501, -1.0732,  2.0400, -0.4386],
                [ 1.7632,  0.7460,  0.0548, -0.8347]])
```

Dù có thể thực hiện lab này bằng estimator `PCA()`, ở đây ta dùng function

`np.linalg.svd()` để minh họa cách sử dụng nó. Bây giờ ta omit 20 entries trong data matrix  $50 \times 4$  một cách random. Ta làm điều này bằng cách trước hết chọn random 20 rows (states), rồi chọn random one trong four entries của mỗi row. Như vậy every row có ít nhất three observed values.

In [26]:

```
n_omit = 20
np.random.seed(15)
r_idx = np.random.choice(np.arange(X.shape[0]),
                          n_omit,
                          replace=False)
c_idx = np.random.choice(np.arange(X.shape[1]),
                          n_omit,
                          replace=True)

Xna = X.copy()
Xna[r_idx, c_idx] = np.nan
```

Array `r_idx` chứa 20 integers từ 0 đến 49, đại diện cho states (rows của  $X$ ) được chọn để có missing values. `c_idx` chứa 20 integers từ 0 đến 3, đại diện cho features (columns trong  $X$ ) có missing values cho từng selected state.

Bây giờ ta viết code để implement Algorithm 12.1. Trước hết viết một function nhận một matrix và returns một approximation của matrix đó bằng function `svd()`. Function này cần cho Step 2 của Algorithm 12.1.

In [27]:

```
def low_rank(X, M=1):
    U, D, V = np.linalg.svd(X)
    L = U[:, :M] * D[None, :M]
    return L.dot(V[:, :M])
```

Để thực hiện Step 1 của algorithm, ta initialize  $\tilde{X}$ at — đây là  $\tilde{X}$  trong Algorithm 12.1 — bằng cách replace missing values bằng column means của non-missing entries. Các means này được lưu trong  $\bar{X}$  sau khi chạy `np.nanmean()` theo row axis. Ta tạo một copy để khi assign values cho  $\tilde{X}$ at bên dưới, ta không overwrite values trong  $X$ na.

*np.nanmean()*

In [28]:

```
Xhat = Xna.copy()
Xbar = np.nanmean(Xhat, axis=0)
Xhat[r_idx, c_idx] = Xbar[c_idx]
```

Trước khi bắt đầu Step 2, ta chuẩn bị để measure progress của iterations:

In [29]:

```
thresh = 1e-7
rel_err = 1
count = 0
ismiss = np.isnan(Xna)
mssold = np.mean(Xhat[~ismiss]**2)
mss0 = np.mean(Xna[~ismiss]**2)
```

Ở đây `ismiss` là một logical matrix có cùng dimensions với  $X$ na; một element là `True` nếu corresponding matrix element bị missing. Notation `~ismiss` negates boolean vector này, nhờ đó ta có thể access cả missing và non-missing entries. Ta lưu mean của squared non-missing elements trong `mss0`. Ta lưu mean squared error của non-missing elements trong old version của  $\tilde{X}$ at vào `mssold`; hiện tại quantity này bằng `mss0`. Ta sẽ lưu mean squared error của non-missing elements trong current version của  $\tilde{X}$ at vào `mss`, rồi iterate Step 2 của Algorithm 12.1 cho đến khi relative

error, defined là  $(mssold - mss) / mss0$ , nhỏ hơn  $thresh = 1e-7$ .<sup>9</sup>

Trong Step 2(a) của Algorithm 12.1, ta approximate  $X_{hat}$  bằng `low_rank()`; gọi kết quả là  $X_{app}$ . Trong Step 2(b), dùng  $X_{app}$  để update estimates cho elements trong  $X_{hat}$  bị missing trong  $X_{na}$ . Cuối cùng, ở Step 2(c), compute relative error. Ba steps này nằm trong while loop sau:

In [30]:

```
while rel_err > thresh:
    count += 1
    # Step 2(a)
    Xapp = low_rank(Xhat, M=1)
    # Step 2(b)
    Xhat[ismiss] = Xapp[ismiss]
    # Step 2(c)
    mss = np.mean((Xna - Xapp)[~ismiss]**2)
    rel_err = (mssold - mss) / mss0
    mssold = mss
    print("Iteration: {0}, MSS :{1:.3f}, Rel.Err {2:.2e}"
          .format(count, mss, rel_err))
```

Out[30]:

```
Iteration: 1, MSS :0.395, Rel.Err 5.99e-01
Iteration: 2, MSS :0.382, Rel.Err 1.33e-02
Iteration: 3, MSS :0.381, Rel.Err 1.44e-03
Iteration: 4, MSS :0.381, Rel.Err 1.79e-04
Iteration: 5, MSS :0.381, Rel.Err 2.58e-05
Iteration: 6, MSS :0.381, Rel.Err 4.22e-06
Iteration: 7, MSS :0.381, Rel.Err 7.65e-07
Iteration: 8, MSS :0.381, Rel.Err 1.48e-07
Iteration: 9, MSS :0.381, Rel.Err 2.95e-08
```

Ta thấy sau eight iterations, relative error đã giảm xuống dưới  $thresh = 1e-7$ , nên algorithm terminates. Khi đó mean squared error của non-missing elements bằng 0.381. Cuối cùng, ta compute correlation giữa 20 imputed values và actual values:

In [31]:

```
np.corrcoef(Xapp[ismiss], X[ismiss])[0,1]
```

Out[31]:

```
0.711
```

Trong lab này, ta tự implement Algorithm 12.1 vì mục đích didactic. Tuy nhiên, reader muốn apply matrix completion lên data của mình có thể tìm các specialized Python implementations.

### 12.5.3 Clustering

#### *K-Means Clustering*

Estimator `sklearn.cluster.KMeans()` thực hiện K-means clustering trong Python. Ta bắt đầu với một simple simulated example trong đó thực sự có two clusters trong data: first 25 observations có một mean shift relative to next 25 observations.

*KMeans()*

<sup>9</sup>Algorithm 12.1 yêu cầu iterate Step 2 đến khi (12.14) không còn decrease. Để xác định (12.14) có decrease hay không, chỉ cần theo dõi  $mssold - mss$ . Trong practice, ta theo dõi  $(mssold - mss) / mss0$ ; cách này làm number of iterations cần để Algorithm 12.1 converge không phụ thuộc vào việc raw data  $X$  có bị multiply bởi một constant factor hay không.

```
In [32]: np.random.seed(0)
X = np.random.standard_normal((50,2))
X[:25,0] += 3
X[:25,1] -= 4
```

Bây giờ thực hiện K-means clustering với  $K = 2$ .

```
In [33]: kmeans = KMeans(n_clusters=2,
                        random_state=2,
                        n_init=20).fit(X)
```

Ta specify `random_state` để results reproducible. Cluster assignments của 50 observations nằm trong `kmeans.labels_`.

```
In [34]: kmeans.labels_
```

```
Out[34]: array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int32)
```

K-means clustering đã separate observations thành two clusters gần như hoàn hảo dù ta không supply bất kỳ group information nào cho `KMeans()`. Ta có thể plot data, coloring mỗi observation theo cluster assignment.

```
In [35]: fig, ax = plt.subplots(1, 1, figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=kmeans.labels_)
ax.set_title("K-Means Clustering Results with K=2")
```

Ở đây observations có thể plot dễ dàng vì chúng two-dimensional. Nếu có nhiều hơn two variables, ta có thể perform PCA rồi plot first two principal component score vectors để represent clusters.

Trong example này ta biết thực sự có two clusters vì chính ta generated data. Với real data, ta không biết true number of clusters, thậm chí không biết chúng có tồn tại theo một precise way hay không. Thay vào đó, ta có thể đã performed K-means clustering với  $K = 3$ .

```
In [36]: kmeans = KMeans(n_clusters=3,
                        random_state=3,
                        n_init=20).fit(X)
fig, ax = plt.subplots(figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=kmeans.labels_)
ax.set_title("K-Means Clustering Results with K=3")
```

Khi  $K = 3$ , K-means clustering split two original clusters. Ta dùng argument `n_init` để run K-means với 20 initial cluster assignments; default là 10. Nếu `n_init > 1`, K-means clustering được performed với multiple random assignments ở Step 1 của Algorithm 12.2, và `KMeans()` chỉ report best result. Ở đây ta compare `n_init=1` với `n_init=20`.

```
In [37]: kmeans1 = KMeans(n_clusters=3,
                        random_state=3,
                        n_init=1).fit(X)
kmeans20 = KMeans(n_clusters=3,
                  random_state=3,
                  n_init=20).fit(X)
kmeans1.inertia_, kmeans20.inertia_
```

Out[37]: (78.06, 75.04)

Lưu ý `kmeans.inertia_` là total within-cluster sum of squares, quantity ta muốn minimize khi thực hiện K-means clustering (12.17). Ta strongly recommend luôn run K-means clustering với large value của `n_init`, chẳng hạn 20 hoặc 50, vì nếu không có thể obtain một undesirable local optimum. Khi perform K-means clustering, ngoài multiple initial cluster assignments, cũng quan trọng phải set random seed bằng argument `random_state` của `KMeans()`. Như vậy initial cluster assignments ở Step 1 có thể replicated và K-means output fully reproducible.

### *Hierarchical Clustering*

Class `AgglomerativeClustering()` từ package `sklearn.cluster` implements hierarchical clustering. Vì tên dài, ta dùng shorthand `HClust`. Điều này không thay return type nên instances vẫn thuộc class `AgglomerativeClustering`. Trong example sau, ta dùng data từ lab trước để plot hierarchical clustering dendrogram với complete, single, và average linkage, dùng Euclidean distance làm dissimilarity measure. Bắt đầu bằng complete linkage.

*Agglomerative-Clustering()*

In [38]:

```
HClust = AgglomerativeClustering
hc_comp = HClust(distance_threshold=0,
                  n_clusters=None,
                  linkage='complete')
hc_comp.fit(X)
```

Lệnh này computes toàn bộ dendrogram. Ta cũng có thể perform hierarchical clustering với average hoặc single linkage:

In [39]:

```
hc_avg = HClust(distance_threshold=0,
                 n_clusters=None,
                 linkage='average')
hc_avg.fit(X)
hc_sing = HClust(distance_threshold=0,
                  n_clusters=None,
                  linkage='single')
hc_sing.fit(X)
```

Để dùng một precomputed distance matrix, ta supply thêm argument `metric='precomputed'`. Trong code dưới, first four lines compute  $50 \times 50$  pairwise-distance matrix.

In [40]:

```
D = np.zeros((X.shape[0], X.shape[0]))
for i in range(X.shape[0]):
    x_ = np.multiply.outer(np.ones(X.shape[0]), X[i])
    D[i] = np.sqrt(np.sum((X - x_)**2, 1))
hc_sing_pre = HClust(distance_threshold=0,
                     n_clusters=None,
                     metric='precomputed',
                     linkage='single')
hc_sing_pre.fit(D)
```

Ta dùng `dendrogram()` từ `scipy.cluster.hierarchy` để plot dendrogram. Tuy nhiên `dendrogram()` expects một so-called linkage-matrix representation của clustering, không được `AgglomerativeClustering()` provide trực tiếp nhưng có thể compute. Function `compute_linkage()` trong

```
dendrogram();
compute_linkage()
```

package `ISLP.cluster` được cung cấp cho mục đích này.

Bây giờ ta có thể plot dendrograms. Numbers ở bottom của plot identify each observation. Function `dendrogram()` có default method để color các branches khác nhau của tree, gợi ý một pre-defined cut tại particular depth. Ta prefer overwrite default này bằng cách set threshold thành infinity. Vì muốn behavior này cho nhiều dendrograms, ta store các values trong dictionary `cargs` rồi pass nó như keyword arguments bằng notation `**cargs`.

```
In [41]: cargs = {'color_threshold':-np.inf,
               'above_threshold_color':'black'}
linkage_comp = compute_linkage(hc_comp)
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_comp,
            ax=ax,
            **cargs)
```

Có thể ta muốn color branches phía trên và dưới một cut-threshold khác nhau. Điều này thực hiện bằng cách change `color_threshold`. Hãy cut tree ở height 4, coloring links merge above 4 bằng black.

```
In [42]: fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_comp,
            ax=ax,
            color_threshold=4,
            above_threshold_color='black')
```

Để determine cluster labels cho mỗi observation associated với một given cut của dendrogram, ta dùng function `cut_tree()` từ `scipy.cluster.hierarchy`:

```
cut_tree()
```

```
In [43]: cut_tree(linkage_comp, n_clusters=4).T
```

```
Out[43]: array([[0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 2, 0, 0, 0, 1, 1, 0, 0, 1,
                  0, 0, 2, 0, 2, 2, 3, 2, 3, 3, 3, 3, 2, 3, 3, 3, 3, 2, 3,
                  3, 3, 3, 2, 3, 3, 3, 3, 3, 3, 3, 3]])
```

Điều này cũng có thể đạt được bằng cách supply argument `n_clusters` cho `HClust()`; tuy nhiên mỗi cut sẽ require recomputing clustering. Tương tự, trees có thể cut theo distance threshold bằng argument `distance_threshold` cho `HClust()` hoặc `height` cho `cut_tree()`.

```
In [44]: cut_tree(linkage_comp, height=5)
```

```
Out[44]: array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
                  0, 0, 1, 0, 1, 1, 2, 1, 2, 2, 2, 2, 1, 2, 2, 2, 2, 1, 2,
                  2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2]])
```

Để scale variables trước khi hierarchical clustering observations, ta dùng `StandardScaler()` như trong PCA example:

```
In [45]: scaler = StandardScaler()
X_scale = scaler.fit_transform(X)
hc_comp_scale = HClust(distance_threshold=0,
                       n_clusters=None,
```

```

linkage='complete').fit(X_scale)
linkage_comp_scale = compute_linkage(hc_comp_scale)
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_comp_scale, ax=ax, **cargs)
ax.set_title("Hierarchical Clustering with Scaled Features")

```

Correlation-based distances giữa observations có thể dùng cho clustering. Correlation giữa two observations measures similarity của their feature values.<sup>10</sup> Với  $n$  observations,  $n \times n$  correlation matrix có thể dùng như similarity (hay affinity) matrix; do đó one minus correlation matrix là dissimilarity matrix dùng cho clustering.

Lưu ý rằng dùng correlation chỉ có ý nghĩa cho data có ít nhất three features, vì absolute correlation giữa any two observations với measurements trên two features luôn bằng one. Vì vậy ta cluster một three-dimensional data set.

In [46]:

```

X = np.random.standard_normal((30, 3))
corD = 1 - np.corrcoef(X)
hc_cor = HClust(linkage='complete',
                distance_threshold=0,
                n_clusters=None,
                metric='precomputed')
hc_cor.fit(corD)
linkage_cor = compute_linkage(hc_cor)
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_cor, ax=ax, **cargs)
ax.set_title("Complete Linkage with Correlation-Based Dissimilarity")

```

### 12.5.4 NCI60 Data Example

Unsupervised techniques thường được dùng trong analysis của genomic data. Đặc biệt, PCA và hierarchical clustering là popular tools. Ta minh họa các techniques này trên NCI60 cancer cell line microarray data, gồm 6830 gene expression measurements trên 64 cancer cell lines.

In [47]:

```

NCI60 = load_data('NCI60')
nci_labs = NCI60['labels']
nci_data = NCI60['data']

```

Mỗi cell line được labeled bằng một cancer type. Ta không dùng cancer types khi perform PCA và clustering, vì đây là unsupervised techniques. Nhưng sau khi thực hiện PCA và clustering, ta sẽ check extent mà các cancer types này agree với results của unsupervised techniques. Data có 64 rows và 6830 columns.

In [48]:

```
nci_data.shape
```

Out[48]: (64, 6830)

Ta bắt đầu bằng examining cancer types của cell lines.

In [49]:

```
nci_labs.value_counts()
```

<sup>10</sup>Giả sử each observation có  $p$  features, mỗi feature là một numerical value. Ta measure similarity của two observations như vậy bằng cách compute correlation của  $p$  pairs of numbers.

```
Out[49]: label
NSCLC      9
RENAL      9
MELANOMA   8
BREAST     7
COLON      7
LEUKEMIA   6
OVARIAN    6
CNS        5
PROSTATE   2
K562A-repro 1
K562B-repro 1
MCF7A-repro 1
MCF7D-repro 1
UNKNOWN    1
dtype: int64
```

### *PCA on the NCI60 Data*

Trước hết ta perform PCA trên data sau khi scale variables (genes) để có standard deviation one, dù ở đây cũng có thể reasonably argue rằng không nên scale genes vì chúng được measured trong cùng units.

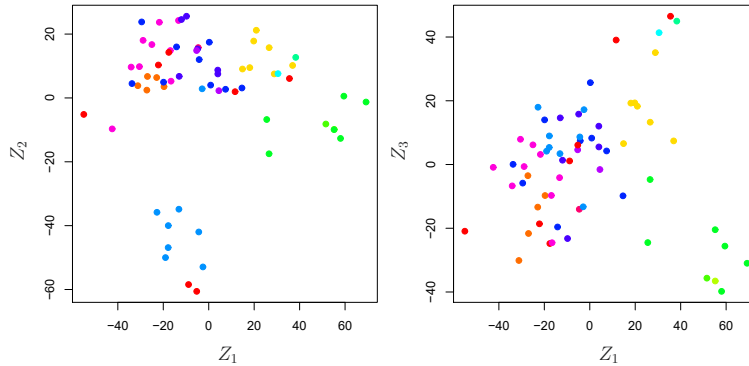
```
In [50]: scaler = StandardScaler()
nci_scaled = scaler.fit_transform(nci_data)
nci_pca = PCA()
nci_scores = nci_pca.fit_transform(nci_scaled)
```

Bây giờ ta plot first few principal component score vectors để visualize data. Observations (cell lines) tương ứng với một given cancer type sẽ được plot cùng color, để xem extent mà observations trong một cancer type similar với nhau.

```
In [51]: cancer_types = list(np.unique(nci_labs))
nci_groups = np.array([cancer_types.index(lab)
                        for lab in nci_labs.values])
fig, axes = plt.subplots(1, 2, figsize=(15,6))
ax = axes[0]
ax.scatter(nci_scores[:,0],
           nci_scores[:,1],
           c=nci_groups,
           marker='o',
           s=50)
ax.set_xlabel('PC1'); ax.set_ylabel('PC2')
ax = axes[1]
ax.scatter(nci_scores[:,0],
           nci_scores[:,2],
           c=nci_groups,
           marker='o',
           s=50)
ax.set_xlabel('PC1'); ax.set_ylabel('PC3')
```

Resulting plots được shown trong Figure 12.17. Nhìn chung, cell lines tương ứng với một single cancer type có xu hướng có similar values trên first few principal component score vectors. Điều này cho thấy cell lines thuộc cùng cancer type thường có khá similar gene expression levels.

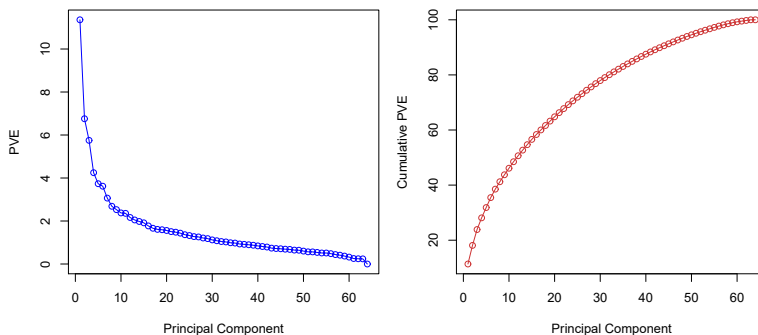
Ta cũng có thể plot percent variance explained bởi principal components cũng như cumulative percent variance explained. Điều này similar với plots đã tạo trước đó cho `USArrests` data.



**HÌNH 12.17.** Projections của NCI60 cancer cell lines lên first three principal components, hay scores của first three principal components. Nhìn chung, observations thuộc cùng một cancer type có xu hướng nằm gần nhau trong low-dimensional space này. Nếu không dùng một dimension reduction method như PCA thì không thể visualize data thuận tiện, vì full data set có  $\binom{6830}{2}$  possible scatterplots và none trong số đó đặc biệt informative.

In [52]:

```
fig, axes = plt.subplots(1, 2, figsize=(15,6))
ax = axes[0]
ticks = np.arange(nci_pca.n_components_)+1
ax.plot(ticks,
        nci_pca.explained_variance_ratio_,
        marker='o')
ax.set_xlabel('Principal Component')
ax.set_ylabel('PVE')
ax = axes[1]
ax.plot(ticks,
        nci_pca.explained_variance_ratio_.cumsum(),
        marker='o')
ax.set_xlabel('Principal Component')
ax.set_ylabel('Cumulative PVE')
```



**HÌNH 12.18.** PVE của principal components của NCI60 cancer cell line microarray data set. Left: PVE của từng principal component. Right: cumulative PVE của principal components. Together, all principal components explain 100% variance.

Resulting plots được shown trong Figure 12.18. Together, first seven

principal components explain khoảng 40% variance trong data. Đây không phải một amount quá lớn. Tuy nhiên, nhìn scree plot, ta thấy mỗi trong first seven principal components giải thích một substantial amount of variance, còn variance explained bởi further principal components giảm rõ rệt. Nói cách khác, có một elbow trong plot sau approximately seventh principal component. Điều này suggests có thể ít benefit khi examine nhiều hơn khoảng seven principal components, dù ngay cả examine seven components cũng có thể khó.

### *Clustering the Observations of the NCI60 Data*

Bây giờ ta perform hierarchical clustering của cell lines trong NCI60 data bằng complete, single, và average linkage. Một lần nữa, goal là tìm xem observations có cluster thành distinct cancer types hay không. Euclidean distance được dùng làm dissimilarity measure. Trước hết ta viết một short function để produce three dendrograms.

In [53]:

```
def plot_nci(linkage, ax, cut=-np.inf):
    cargs = {'above_threshold_color':'black',
            'color_threshold':cut}
    hc = HClust(n_clusters=None,
               distance_threshold=0,
               linkage=linkage.lower()).fit(nci_scaled)
    linkage_ = compute_linkage(hc)
    dendrogram(linkage_,
               ax=ax,
               labels=np.asarray(nci_labs),
               leaf_font_size=10,
               **cargs)
    ax.set_title('%s Linkage' % linkage)
    return hc
```

Hãy plot results.

In [54]:

```
fig, axes = plt.subplots(3, 1, figsize=(15,30))
ax = axes[0]; hc_comp = plot_nci('Complete', ax)
ax = axes[1]; hc_avg = plot_nci('Average', ax)
ax = axes[2]; hc_sing = plot_nci('Single', ax)
```

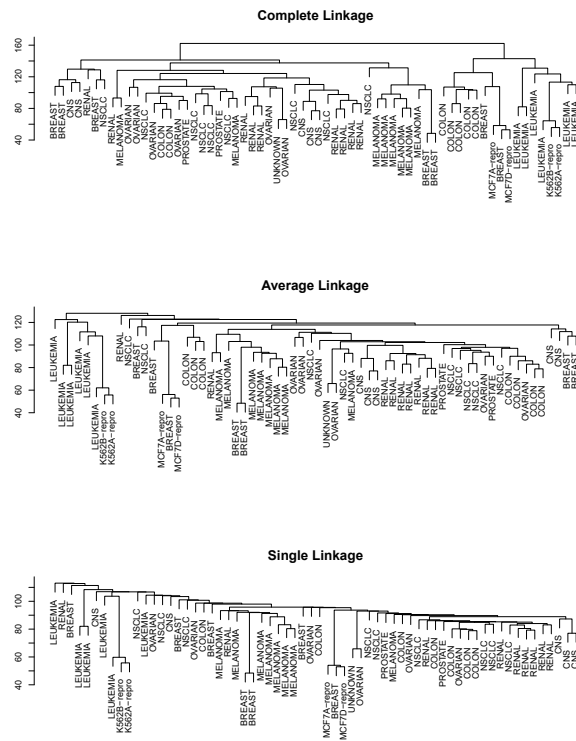
Results shown trong Figure 12.19 cho thấy choice of linkage thực sự affect results. Typically, single linkage tend to yield trailing clusters: very large clusters mà individual observations attach one-by-one. Ngược lại, complete và average linkage tend to yield more balanced, attractive clusters. Vì lý do này, complete và average linkage thường được preferred hơn single linkage. Rõ ràng cell lines trong một single cancer type có xu hướng cluster together, dù clustering không perfect. Ta sẽ dùng complete linkage hierarchical clustering cho analysis tiếp theo.

Ta có thể cut dendrogram ở height cho ra một particular number of clusters, chẳng hạn four:

In [55]:

```
linkage_comp = compute_linkage(hc_comp)
comp_cut = cut_tree(linkage_comp, n_clusters=4).reshape(-1)
pd.crosstab(nci_labs['label'],
            pd.Series(comp_cut.reshape(-1), name='Complete'))
```

Có một số clear patterns. All leukemia cell lines rơi vào one cluster, trong khi breast cancer cell lines spread trên three different clusters. Ta có thể



**HÌNH 12.19.** NCI60 cancer cell line microarray data, clustered bằng average, complete, và single linkage, dùng Euclidean distance làm dissimilarity measure. Complete và average linkage có xu hướng yield evenly sized clusters, trong khi single linkage có xu hướng yield extended clusters mà individual observations được fused one by one.

plot một cut trên dendrogram produce four clusters này:

```
In [56]: fig, ax = plt.subplots(figsize=(10,10))
          plot_nci('Complete', ax, cut=140)
          ax.axhline(140, c='r', linewidth=4)
```

Function `axhline()` draws một horizontal line trên existing axes. Argument 140 plot line tại height 140 trên dendrogram; đây là height result in four distinct clusters. Có thể verify rằng resulting clusters giống các clusters obtained trong `comp_cut`.

Trước đó ở Section 12.4.2 ta đã nói K-means clustering và hierarchical clustering với dendrogram cut để obtain cùng number of clusters có thể yield very different results. NCI60 hierarchical clustering results này compare thế nào với K-means clustering khi  $K = 4$ ?

```
In [57]: nci_kmeans = KMeans(n_clusters=4,
                             random_state=0,
                             n_init=20).fit(nci_scaled)
          pd.crosstab(pd.Series(comp_cut, name='HClust'),
                     pd.Series(nci_kmeans.labels_, name='K-means'))
```

```
Out[57]: K-means    0  1  2  3
          HClust
0          28  3  9  0
1           7  0  0  0
2           0  0  0  8
3           0  9  0  0
```

Ta thấy four clusters obtained bằng hierarchical clustering và K-means clustering somewhat different. Trước hết cluster labels trong two clusterings là arbitrary: swapping identifier của cluster không change clustering. Ở đây Cluster 3 trong K-means clustering identical với cluster 2 trong hierarchical clustering. Nhưng các clusters khác differ; chẳng hạn cluster 0 trong K-means chứa một phần observations assigned to cluster 0 bởi hierarchical clustering, cũng như all observations assigned to cluster 1 bởi hierarchical clustering.

Thay vì hierarchical clustering trên entire data matrix, ta cũng có thể perform hierarchical clustering trên first few principal component score vectors, coi các first few components này như một less noisy version of data.

```
In [58]: hc_pca = HClust(n_clusters=None,
                        distance_threshold=0,
                        linkage='complete').fit(nci_scores[:, :5])
          linkage_pca = compute_linkage(hc_pca)
          fig, ax = plt.subplots(figsize=(8,8))
          dendrogram(linkage_pca,
                     labels=np.asarray(nci_labs),
                     leaf_font_size=10,
                     ax=ax,
                     **cargs)
          ax.set_title("Hier. Clust. on First Five Score Vectors")
          pca_labels = pd.Series(cut_tree(linkage_pca,
                                         n_clusters=4).reshape(-1),
                               name='Complete-PCA')
          pd.crosstab(nci_labs['label'], pca_labels)
```

## 12.6 Exercises

### Conceptual

- Bài này liên quan đến K-means clustering algorithm.
  - Chúng minh (12.18).
  - Dựa trên identity này, hãy lập luận rằng K-means clustering algorithm (Algorithm 12.2) decreases objective (12.17) ở mỗi iteration.
- Giả sử ta có four observations, với dissimilarity matrix

$$\begin{bmatrix} 0 & 0.3 & 0.4 & 0.7 \\ 0.3 & 0 & 0.5 & 0.8 \\ 0.4 & 0.5 & 0 & 0.45 \\ 0.7 & 0.8 & 0.45 & 0 \end{bmatrix}.$$

Chẳng hạn, dissimilarity giữa first và second observations là 0.3, và giữa second và fourth observations là 0.8.

- Dựa trên dissimilarity matrix này, sketch dendrogram resulting from hierarchically clustering four observations bằng complete linkage. Hãy indicate height của mỗi fusion và observations corresponding to mỗi leaf.
  - Repeat (a), lần này dùng single linkage clustering.
  - Giả sử cut dendrogram trong (a) để result in two clusters. Observations nào thuộc từng cluster?
  - Giả sử cut dendrogram trong (b) để result in two clusters. Observations nào thuộc từng cluster?
  - Chapter đã nói rằng tại each fusion trong dendrogram, position của two clusters being fused có thể swap mà không change meaning. Draw một dendrogram equivalent với dendrogram ở (a), trong đó two or more leaves được repositioned nhưng meaning không đổi.
- Trong bài này, bạn sẽ perform K-means clustering manually với  $K = 2$  trên small example có  $n = 6$  observations và  $p = 2$  features. Observations như sau.

| Obs. | $X_1$ | $X_2$ |
|------|-------|-------|
| 1    | 1     | 4     |
| 2    | 1     | 3     |
| 3    | 0     | 4     |
| 4    | 5     | 1     |
| 5    | 6     | 2     |
| 6    | 4     | 0     |

- Plot observations.
- Randomly assign một cluster label cho mỗi observation. Có thể dùng `np.random.choice()`. Report cluster labels cho each observation.

- (c) Compute centroid của mỗi cluster.
  - (d) Assign mỗi observation cho centroid gần nhất theo Euclidean distance. Report cluster labels.
  - (e) Repeat (c) và (d) đến khi answers stop changing.
  - (f) Trong plot ở (a), color observations theo cluster labels obtained.
4. Giả sử với một particular data set, ta perform hierarchical clustering bằng single linkage và complete linkage, obtaining two dendrograms.
- (a) Tại một point trên single linkage dendrogram, clusters  $\{1, 2, 3\}$  và  $\{4, 5\}$  fuse. Trên complete linkage dendrogram, same clusters cũng fuse. Fusion nào occur higher trên tree, chúng occur at same height, hay không đủ information để biết?
  - (b) Tại một point trên single linkage dendrogram, clusters  $\{5\}$  và  $\{6\}$  fuse. Trên complete linkage dendrogram, same clusters cũng fuse. Fusion nào occur higher, same height, hay không đủ information?
5. Bằng lời, describe results bạn expect nếu perform K-means clustering của eight shoppers trong Figure 12.16 dựa trên sock và computer purchases với  $K = 2$ . Give three answers, one cho mỗi variable scaling displayed. Explain.
6. Ta đã thấy ở Section 12.2.2 rằng principal component loading và score vectors provide một approximation cho matrix theo sense của (12.5). Cụ thể, principal component score và loading vectors solve optimization problem (12.6).  
 Bây giờ giả sử  $M$  principal component score vectors  $z_{im}$ ,  $m = 1, \dots, M$ , đã known. Dùng (12.6), explain rằng mỗi trong first  $M$  principal component loading vectors  $\phi_{jm}$ ,  $m = 1, \dots, M$ , có thể obtain bằng cách perform  $p$  separate least squares linear regressions. Trong each regression, principal component score vectors là predictors và one feature của data matrix là response.

### Applied

7. Trong chapter này, ta đã nhắc đến correlation-based distance và Euclidean distance như dissimilarity measures cho hierarchical clustering. Hai measures này gần như equivalent: nếu mỗi observation đã centered để mean zero và standard deviation one, và  $r_{ij}$  là correlation giữa  $i$ th và  $j$ th observations, thì quantity  $1 - r_{ij}$  proportional với squared Euclidean distance giữa hai observations.  
 Trên USArrests data, show proportionality này. *Hint:* Euclidean distance có thể calculate bằng `pairwise_distances()` từ module `sklearn.metrics`, còn correlations bằng `np.corrcoef()`.
8. Ở Section 12.2.3, formula để calculate PVE được cho trong Equation 12.10. Ta cũng thấy PVE có thể obtain bằng attribute `explained_variance_ratio_` của fitted `PCA()` estimator. Trên USArrests data, calculate PVE theo two ways:
- (a) Dùng `explained_variance_ratio_` output của fitted `PCA()` estimator, như trong Section 12.2.3.

*pairwise-  
distances()*

- (b) Apply Equation 12.10 trực tiếp. Loadings được lưu trong attribute `components_` của fitted `PCA()` estimator. Dùng loadings này trong Equation 12.10 để obtain PVE.

Hai approaches nên give same results. *Hint:* Chỉ obtain same results ở (a) và (b) nếu same data được used trong cả hai. Chẳng hạn, nếu ở (a) performed `PCA()` với centered và scaled variables thì phải center và scale variables trước khi apply Equation 12.10 ở (b).

9. Xét `USArrests` data. Bây giờ ta perform hierarchical clustering trên states.
  - (a) Dùng hierarchical clustering với complete linkage và Euclidean distance, cluster states.
  - (b) Cut dendrogram tại một height result in three distinct clusters. States nào thuộc từng cluster?
  - (c) Hierarchically cluster states bằng complete linkage và Euclidean distance sau khi scale variables để có standard deviation one.
  - (d) Scaling variables ảnh hưởng thế nào đến hierarchical clustering obtained? Theo bạn, variables có nên scaled trước khi compute inter-observation dissimilarities không? Justify answer.
10. Trong bài này, bạn sẽ generate simulated data rồi perform PCA và K-means clustering.
  - (a) Generate simulated data set với 20 observations trong mỗi of three classes, tức 60 observations total, và 50 variables. *Hint:* Có nhiều functions trong Python để generate data, chẳng hạn method `normal()` của `random()` trong numpy; `uniform()` là option khác. Hãy add mean shift vào observations trong each class để có three distinct classes.
  - (b) Perform PCA trên 60 observations và plot first two principal component score vectors. Dùng different color để indicate observations trong three classes. Nếu classes appear separated thì continue (c); nếu không, return (a) và modify simulation để separation lớn hơn. Không continue đến (c) cho đến khi three classes show at least some separation trong first two score vectors.
  - (c) Perform K-means clustering với  $K = 3$ . Clusters obtained compare thế nào với true class labels? *Hint:* Có thể dùng `pd.crosstab()` để compare labels. Hãy cẩn thận khi interpret: K-means arbitrarily numbers clusters, nên không thể simply check true labels equal clustering labels.
  - (d) Perform K-means clustering với  $K = 2$ . Describe results.
  - (e) Perform K-means clustering với  $K = 4$ , và describe results.
  - (f) Perform K-means clustering với  $K = 3$  trên first two principal component score vectors thay vì raw data; tức cluster  $60 \times 2$  matrix whose first column là first PC score vector và second column là second PC score vector. Comment on results.
  - (g) Dùng `StandardScaler()` estimator, perform K-means clustering với  $K = 3$  sau khi scale each variable để standard deviation one.

Results compare thế nào với (b)? Explain.

11. Viết một Python function để perform matrix completion như Algorithm 12.1 và như outlined trong Section 12.5.2. Trong mỗi iteration, function phải keep track relative error và iteration count. Iterations tiếp tục cho đến khi relative error đủ small hoặc đạt some maximum number of iterations; hãy set một default value cho maximum. Ngoài ra phải có option print progress ở mỗi iteration.  
Test function trên **Boston** data. Trước hết standardize features để mean zero và standard deviation one bằng **StandardScaler()**. Run experiment trong đó randomly leave out an increasing và nested number of observations từ 5% đến 30%, step 5%. Apply Algorithm 12.1 với  $M = 1, 2, \dots, 8$ . Display approximation error như function của fraction observations missing và value of  $M$ , averaged over 10 repetitions.
12. Trong Section 12.5.2, Algorithm 12.1 được implemented bằng function **svd()** từ module **np.linalg**. Tuy nhiên, dựa trên connection giữa **svd()** và **PCA()** highlighted trong lab, ta có thể implement algorithm bằng **PCA()** thay thế. Viết một function implement Algorithm 12.1 dùng **PCA()** thay vì **svd()**.
13. Trên book website, [www.statlearning.com](http://www.statlearning.com), có gene expression data set **Ch12Ex13.csv** gồm 40 tissue samples với measurements trên 1,000 genes. First 20 samples từ healthy patients, second 20 từ diseased group.
  - (a) Load data bằng **pd.read\_csv()**. Cần select **header=None**.
  - (b) Apply hierarchical clustering cho samples bằng correlation-based distance và plot dendrogram. Genes có separate samples thành two groups không? Results có depend on linkage type không?
  - (c) Collaborator muốn biết genes nào differ nhiều nhất across two groups. Suggest một way để answer và apply ở đây.

# 13

## Multiple Testing

Cho đến đây, textbook chủ yếu tập trung vào estimation và người họ hàng gần của nó là prediction. Trong chapter này, ta chuyển sang hypothesis testing, một thành phần then chốt của inference. Nhắc lại rằng inference đã được discuss ngắn gọn ở Chapter 2.

Section 13.1 cung cấp một brief review về null hypotheses, p-values, test statistics, và các key ideas khác trong hypothesis testing. Tuy nhiên, chapter này assumes reader đã từng tiếp xúc với các topics đó. Cụ thể, ta sẽ không tập trung vào lý do tại sao hoặc cách conduct một hypothesis test — một topic mà cả cuốn sách có thể, và đã, được viết về nó. Thay vào đó, ta assume reader muốn test một particular set of null hypotheses và đã có một specific plan về cách conduct tests và obtain p-values.

Phần lớn emphasis trong classical statistics tập trung vào testing một single null hypothesis. Chẳng hạn, ta có thể test  $H_0$ : expected blood pressure của mice trong control group bằng expected blood pressure của mice trong treatment group. Dĩ nhiên, ta có thể hy vọng discover rằng mean blood pressure của two groups differs. Nhưng vì reasons sẽ sớm rõ, ta construct null hypothesis tương ứng với no difference.

Trong contemporary settings, ta thường đối mặt với huge amounts of data và vì thế có thể muốn test rất nhiều null hypotheses. Chẳng hạn, thay vì chỉ test  $H_0$ , ta có thể muốn test  $m$  null hypotheses  $H_{01}, \dots, H_{0m}$ , trong đó  $H_{0j}$  states rằng expected value của  $j$ th biomarker trong control group bằng expected value của  $j$ th biomarker trong treatment group. Khi conduct multiple testing, ta cần rất careful về cách interpret results để tránh erroneously rejecting far too many null hypotheses.

Chapter này discusses cả classical lẫn more contemporary ways để conduct multiple testing trong big-data setting. Section 13.2 highlights challenges associated với multiple testing. Classical solutions cho các challenges này được trình bày ở Section 13.3, còn more contemporary solutions ở Sections 13.4 và 13.5.

Đặc biệt, Section 13.4 tập trung vào false discovery rate. Notion này có từ thập niên 1990 và nhanh chóng trở nên popular vào đầu những năm 2000, khi large-scale datasets bắt đầu xuất hiện trong genomics. Các datasets đó đặc biệt không chỉ vì size lớn,<sup>1</sup> mà còn vì chúng thường được collected cho exploratory purposes: researchers collected chúng để test một huge number

---

<sup>1</sup>Microarray data từng được xem là “big data” vào thời điểm đó, dù theo standards ngày nay label này nghe khá khiêm tốn: một khá khiếm tốn: một microarray dataset có thể, và thường đã, được lưu trong một Microsoft Excel spreadsheet.

of null hypotheses, thay vì chỉ một very small number of pre-specified null hypotheses. Ngày nay, huge datasets được collected mà không có pre-specified null hypothesis trong gần như every field. Như ta sẽ thấy, false discovery rate rất well-suited với modern-day reality này.

Chapter này naturally centers on classical statistical technique của p-values, được dùng để quantify results của hypothesis tests. Vào thời điểm cuốn sách được viết (2020), p-values gần đây đã trở thành topic của extensive commentary trong social science research community, đến mức một số social science journals thậm chí ban việc dùng p-values. Ta chỉ comment rằng, khi được understand và apply đúng, p-values là một powerful tool để draw inferential conclusions từ data.

## 13.1 A Quick Review of Hypothesis Testing

Hypothesis tests provide một rigorous statistical framework để answer các simple “yes-or-no” questions về data, chẳng hạn:

1. True coefficient  $\beta_j$  trong linear regression của  $Y$  lên  $X_1, \dots, X_p$  có bằng zero không?<sup>2</sup>
2. Có difference trong expected blood pressure của laboratory mice ở control group và laboratory mice ở treatment group không?<sup>3</sup>

Trong Section 13.1.1, ta briefly review các steps involved trong hypothesis testing. Section 13.1.2 discusses different types of mistakes, hay errors, có thể occur trong hypothesis testing.

### 13.1.1 Testing a Hypothesis

Conducting một hypothesis test thường proceed qua four steps. First, define null và alternative hypotheses. Next, construct một test statistic summarize strength of evidence against null hypothesis. Sau đó compute p-value quantifies probability of having obtained một comparable hoặc more extreme value của test statistic under null hypothesis. Finally, dựa trên p-value, decide whether to reject null hypothesis. Ta briefly discuss từng step.

#### Step 1: Define the Null and Alternative Hypotheses

Trong hypothesis testing, ta divide world thành two possibilities: null hypothesis và alternative hypothesis. Null hypothesis, denoted  $H_0$ , là default state of belief về world.<sup>4</sup> Chẳng hạn, null hypotheses corresponding với two questions posed earlier là:

1. True coefficient  $\beta_j$  trong linear regression của  $Y$  lên  $X_1, \dots, X_p$  equals zero.
2. Không có difference giữa expected blood pressure của mice trong control và treatment groups.

null hypothesis;  
alternative  
hypothesis

<sup>2</sup>Hypothesis test này đã được discuss ở page 76 của Chapter 3.

<sup>3</sup>“Treatment group” là set of mice nhận experimental treatment, còn “control group” là những con không nhận treatment.

<sup>4</sup> $H_0$  được phát âm là “H naught” hoặc “H zero”.

Null hypothesis “boring” by construction: nó hoàn toàn có thể true, nhưng ta có thể hy vọng data sẽ tell us otherwise. Alternative hypothesis, denoted  $H_a$ , represents something different và unexpected; chẳng hạn expected blood pressure của mice trong two groups differs. Typically, alternative hypothesis simply posits rằng null hypothesis does not hold: nếu null nói A và B không khác nhau, alternative nói chúng khác nhau.

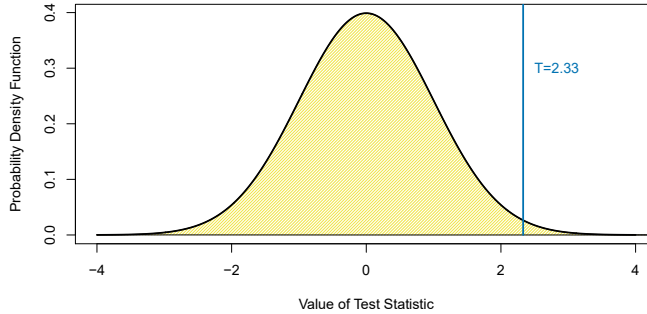
Điều quan trọng là treatment của  $H_0$  và  $H_a$  asymmetric.  $H_0$  được treated như default state of world, và ta focus on using data to reject  $H_0$ . Nếu reject  $H_0$ , điều đó provides evidence in favor of  $H_a$ . Có thể think of rejecting  $H_0$  như making a discovery about data: ta discover rằng  $H_0$  does not hold. By contrast, nếu fail to reject  $H_0$ , findings more nebulous: ta không biết có phải sample size quá small — trong trường hợp đó testing lại trên larger hoặc higher-quality dataset có thể lead to rejection — hay vì  $H_0$  thực sự holds.

### Step 2: Construct the Test Statistic

Tiếp theo, ta muốn use data để find evidence for or against null hypothesis. Để làm vậy, ta compute một test statistic, denoted  $T$ , summarize extent mà data consistent với  $H_0$ . Cách construct  $T$  depends on nature của null hypothesis đang test.

Để concrete, let  $x_1^t, \dots, x_{n_t}^t$  là blood pressure measurements cho  $n_t$  mice trong treatment group, và  $x_1^c, \dots, x_{n_c}^c$  cho  $n_c$  mice trong control group; đặt  $\mu_t = E(X^t)$  và  $\mu_c = E(X^c)$ . Để test  $H_0 : \mu_t = \mu_c$ , ta dùng two-sample t-statistic.<sup>5</sup>

test statistic;  
two-sample  
t-statistic



**HÌNH 13.1.** Density function của  $N(0,1)$  distribution, với vertical line tại value 2.33. Có 1% area under curve nằm bên phải line, nên chỉ có 2% chance quan sát một  $N(0,1)$  value lớn hơn 2.33 hoặc nhỏ hơn  $-2.33$ . Vì vậy, nếu một test statistic có  $N(0,1)$  null distribution, observed test statistic  $T = 2.33$  dẫn đến p-value 0.02.

$$T = \frac{\hat{\mu}_t - \hat{\mu}_c}{s \sqrt{\frac{1}{n_t} + \frac{1}{n_c}}}, \quad (13.1)$$

where  $\hat{\mu}_t = n_t^{-1} \sum_{i=1}^{n_t} x_i^t$ ,  $\hat{\mu}_c = n_c^{-1} \sum_{i=1}^{n_c} x_i^c$ , and

$$s = \sqrt{\frac{(n_t - 1)s_t^2 + (n_c - 1)s_c^2}{n_t + n_c - 2}}, \quad (13.2)$$

<sup>5</sup>T-statistic lấy tên từ fact rằng dưới  $H_0$ , nó follows một t-distribution.

is an estimator of pooled standard deviation của two samples.<sup>6</sup> Ở đây  $s_t^2$  và  $s_c^2$  là unbiased estimators của variance của blood pressure trong treatment và control groups. Một large absolute value của  $T$  provides evidence against  $H_0 : \mu_t = \mu_c$ , và therefore evidence in support of  $H_a : \mu_t \neq \mu_c$ .

### Step 3: Compute the p-Value

Ở previous section, ta noted rằng large absolute value của two-sample t-statistic provides evidence against  $H_0$ . Câu hỏi ngay lập tức là: large đến mức nào mới là large? Nói cách khác, một given value của test statistic provides bao nhiêu evidence against  $H_0$ ?

Notion của p-value cho ta cách formalize và answer question này. P-value được defined là probability of observing một test statistic equal to or more extreme than observed statistic, under assumption rằng  $H_0$  thật sự true. Vì vậy small p-value provides evidence against  $H_0$ .

Để concrete, suppose  $T = 2.33$  cho test statistic trong (13.1). Ta ask: nếu  $H_0$  holds, probability quan sát such a large value của  $T$  là bao nhiêu? Under  $H_0$ , distribution của  $T$  trong (13.1) approximately follows một  $N(0, 1)$  distribution<sup>7</sup> — normal distribution với mean 0 và variance 1. Figure 13.1 shows distribution này. Có 98% của  $N(0, 1)$  distribution nằm between  $-2.33$  và  $2.33$ , nghĩa là under  $H_0$  ta expect see such a large  $|T|$  chỉ 2% of the time. Vì vậy p-value corresponding to  $T = 2.33$  là 0.02.

Distribution của test statistic under  $H_0$ , còn gọi là null distribution của test statistic, depends on details của type of null hypothesis và type of test statistic. Nói chung, most commonly-used test statistics follow một well-known statistical distribution under null hypothesis — chẳng hạn normal, t-,  $\chi^2$ -, hoặc F-distribution — provided sample size sufficiently large và some other assumptions hold. Typically, R function dùng để compute test statistic sẽ use null distribution này để output p-value. Trong Section 13.5 ta sẽ thấy một approach estimate null distribution bằng re-sampling; trong many contemporary settings, đây là attractive option vì uses fast computers để avoid potentially problematic assumptions về data.

P-value có lẽ là một trong các notions được dùng và abused nhiều nhất trong statistics. Đôi khi người ta nói p-value là probability that  $H_0$  holds, tức probability null hypothesis true. Điều này incorrect. One and only correct interpretation của p-value là fraction of the time ta expect see such an extreme value của test statistic nếu repeated experiment many, many times, provided  $H_0$  holds.<sup>8</sup>

Trong Step 2 ta computed test statistic và noted large absolute value provides evidence against  $H_0$ . Trong Step 3, test statistic converted thành p-value, với small p-values providing evidence against  $H_0$ . Vậy conversion

<sup>6</sup>Equation (13.2) assumes control và treatment groups have equal variance. Without assumption này, (13.2) có một slightly different form.

<sup>7</sup>Precisely hơn, assuming observations drawn from a normal distribution,  $T$  follows a t-distribution với  $n_t + n_c - 2$  degrees of freedom. Provided  $n_t + n_c - 2$  larger than around 40, distribution này very well approximated by  $N(0, 1)$ . Trong Section 13.5 ta sẽ thấy alternative, thường attractive hơn, để approximate null distribution của  $T$  mà không cần stringent assumptions về data.

<sup>8</sup>Một one-sided p-value là probability of seeing such an extreme value của test statistic, ví dụ probability of a test statistic greater than or equal to  $T = 2.33$ . Một two-sided p-value là probability of seeing such an extreme absolute test statistic, ví dụ greater than or equal to 2.33 hoặc less than or equal to  $-2.33$ . Default recommendation là report two-sided p-value trừ khi có clear and compelling reason chỉ một direction có scientific interest.

p-value

null distribution

này accomplish gì? Suppose data analyst conducts statistical test và reports  $T = 17.3$ . Value này provide strong evidence against  $H_0$  không? Không thể biết without more information, đặc biệt cần biết expected value của test statistic under  $H_0$ . Chính p-value cho ta information đó. Nói cách khác, p-value transforms test statistic từ một arbitrary và hard-to-interpret scale thành number between 0 and 1 để interpret hơn.

#### Step 4: Decide Whether to Reject the Null Hypothesis

Sau khi compute p-value corresponding to  $H_0$ , ta cần decide whether to reject  $H_0$ . Ta thường không nói “accepting”  $H_0$ ; thay vào đó nói “failing to reject”  $H_0$ . Small p-value indicates such a large test statistic unlikely under  $H_0$  và thereby provides evidence against it. Nếu p-value sufficiently small, ta muốn reject  $H_0$ , tức make a “discovery”. Nhưng small đến mức nào mới đủ?

Answer depends heavily on data analyst. Smaller p-value means stronger evidence against  $H_0$ . Trong some fields, typical rule là reject  $H_0$  nếu p-value below 0.05, nghĩa là nếu  $H_0$  holds, ta expect such a small p-value no more than 5% of the time.<sup>9</sup> Trong other fields, burden of proof much higher: chẳng hạn some areas of physics chỉ reject  $H_0$  nếu p-value below  $10^{-9}$ .

Trong example Figure 13.1, nếu dùng threshold 0.05 làm cut-off thì reject null; nếu threshold 0.01 thì fail to reject. Các ideas này được formalize trong next section.

#### 13.1.2 Type I and Type II Errors

Nếu null hypothesis holds, ta gọi nó là true null hypothesis; otherwise là false null hypothesis. Chẳng hạn, nếu test  $H_0 : \mu_t = \mu_c$  như Section 13.1.1 và thật sự không có difference trong population mean blood pressure giữa mice treatment và control groups,  $H_0$  true; otherwise false. Dĩ nhiên a priori ta không biết  $H_0$  true hay false; đó là lý do cần hypothesis test.

true null  
hypothesis; false  
null hypothesis

| Decision            | Truth        |               |
|---------------------|--------------|---------------|
|                     | $H_0$        | $H_a$         |
| Reject $H_0$        | Type I Error | Correct       |
| Do Not Reject $H_0$ | Correct      | Type II Error |

**BẢNG 13.1.** Summary của possible scenarios associated với testing null hypothesis  $H_0$ . Type I errors còn gọi là false positives, và Type II errors là false negatives.

Table 13.1 summarizes possible scenarios associated với testing  $H_0$ .<sup>10</sup> Sau hypothesis test, row của table known dựa trên whether ta rejected  $H_0$ ; nhưng impossible biết column nào là truth. Nếu reject  $H_0$  khi false, tức  $H_a$  true, hoặc do not reject  $H_0$  khi true, ta arrive at correct result. Nếu erroneously reject  $H_0$  khi actually true, ta commit Type I error. Type I error rate là probability of making a Type I error given  $H_0$  holds, tức

<sup>9</sup>Threshold 0.05 để reject  $H_0$  rất ubiquitous trong some areas of science, nhưng authors advise against blind adherence to arbitrary choice này. Data analyst typically nên report p-value itself thay vì chỉ report whether nó exceeds một threshold.

<sup>10</sup>Có parallels giữa Table 13.1 và Table 4.6 về output của binary classifier. Recall rằng false positive từ Table 4.6 results from predicting positive (non-null) label khi true label actually negative (null). Điều này closely related to Type I error, xảy ra khi rejecting null hypothesis trong khi null actually holds.

probability of incorrectly rejecting  $H_0$ . Nếu do not reject  $H_0$  khi actually false, ta commit Type II error. Power của hypothesis test là probability of not making Type II error given  $H_a$  holds, tức probability of correctly rejecting  $H_0$ .

Type I error; Type  
I error rate; Type  
II error; power

Ideally, ta muốn cả Type I và Type II error rates small. Nhưng trong practice khó đạt cả hai. Usually có trade-off: có thể make Type I error small bằng cách chỉ reject  $H_0$  khi very sure it does not hold, nhưng điều đó increases Type II error. Hoặc make Type II error small bằng cách reject  $H_0$  ngay khi có modest evidence against it, nhưng Type I error sẽ large. Trong practice, Type I errors thường được xem serious hơn Type II errors vì former involves declaring một scientific finding không correct. Vì vậy hypothesis testing thường require low Type I error rate, ví dụ at most  $\alpha = 0.05$ , trong khi cố make Type II error small, equivalently power large.

Có direct correspondence giữa p-value threshold causing rejection của  $H_0$  và Type I error rate. Nếu chỉ reject  $H_0$  khi p-value below  $\alpha$ , ta ensure Type I error rate less than or equal to  $\alpha$ .

## 13.2 The Challenge of Multiple Testing

Ở previous section, ta thấy reject  $H_0$  khi p-value below, say, 0.01 cho một simple way to control Type I error cho  $H_0$  at level 0.01: nếu  $H_0$  true, probability reject nó no more than 1%. Nhưng suppose ta muốn test  $m$  null hypotheses  $H_{01}, \dots, H_{0m}$ . Có đủ chỉ reject tất cả null hypotheses có corresponding p-value below 0.01 không? Nói cách khác, nếu reject tất cả hypotheses có p-value below 0.01, ta expect make bao nhiêu Type I errors?

Bước đầu để answer question, consider một stockbroker muốn attract new clients bằng cách convince họ rằng she has trading acumen. She tells 1,024 potential clients —  $1,024 = 2^{10}$  — rằng có thể correctly predict whether Apple's stock price will increase or decrease trong 10 days liên tiếp. Có  $2^{10}$  possibilities cho sequence tăng/giảm qua 10 days. Vì vậy she emails mỗi client one trong  $2^{10}$  possibilities. Vast majority sẽ thấy predictions no better than chance, thậm chí worse. Nhưng a broken clock is right twice a day, và one potential client sẽ very impressed vì predictions đúng all 10 days, nên stockbroker gains a new client.

Điều gì happened? Stockbroker có actual insight nào về Apple stock price không? Không. Vậy làm sao she predicted perfectly trong 10 days? Answer là she made a lot of guesses và one guess happened to be exactly right.

Điều này liên quan multiple testing thế nào? Suppose ta flip 1,024 fair coins<sup>11</sup> ten times each. Ta expect on average one coin come up all tails. Probability một individual coin cho all tails là  $1/2^{10} = 1/1,024$ , nên với 1,024 coins ta expect one all-tails coin on average. Nếu one coin all tails, ta có thể conclude particular coin không fair. In fact, một standard hypothesis test cho null rằng coin fair cho p-value below 0.002.<sup>12</sup> Nhưng conclude coin unfair sẽ incorrect: null hypothesis actually holds; ta chỉ happened to get ten tails in a row by chance.

<sup>11</sup>Fair coin là coin có equal chance landing heads hoặc tails.

<sup>12</sup>Recall p-value là probability of observing data at least this extreme under null. Nếu coin fair, probability observing at least ten tails là  $(1/2)^{10} = 1/1,024 < 0.001$ . P-value therefore  $2/1,024 < 0.002$ , vì đây là probability of observing ten heads hoặc ten tails.

Các examples này illustrate main challenge của multiple testing: khi testing huge number of null hypotheses, ta bound to get some very small p-values by chance. Nếu decide whether to reject each null independently mà không account for fact rằng đã performed very large number of tests, ta có thể reject great number of true null hypotheses — tức make many Type I errors.

multiple testing

Problem severe đến đâu? Recall nếu reject single  $H_0$  khi p-value less than, say,  $\alpha = 0.01$ , thì có 1% chance of false rejection if  $H_0$  true. Bây giờ nếu test  $m$  null hypotheses  $H_{01}, \dots, H_{0m}$ , all true, thì mỗi individual null có 1% chance bị rejected; do đó expect falsely reject approximately  $0.01 \times m$  null hypotheses. Nếu  $m = 10,000$ , expect falsely reject 100 hypotheses by chance. Đó là rất nhiều Type I errors.

Crux của issue là: rejecting một null hypothesis khi p-value below  $\alpha$  controls probability of falsely rejecting that particular null at level  $\alpha$ . Tuy nhiên nếu làm vậy cho  $m$  null hypotheses, chance of falsely rejecting at least one trong  $m$  hypotheses sẽ much higher.

|                     | $H_0$ is True | $H_0$ is False | Total   |
|---------------------|---------------|----------------|---------|
| Reject $H_0$        | $V$           | $S$            | $R$     |
| Do Not Reject $H_0$ | $U$           | $W$            | $m - R$ |
| Total               | $m_0$         | $m - m_0$      | $m$     |

**BẢNG 13.2.** Summary của results khi testing  $m$  null hypotheses. Một given null hypothesis either true or false, và một test có thể reject hoặc fail to reject nó. Trong practice, individual values của  $V$ ,  $S$ ,  $U$ , và  $W$  unknown. Tuy nhiên ta observe  $V + S = R$  và  $U + W = m - R$ , là numbers of null hypotheses rejected và not rejected.

Ta sẽ investigate issue này chi tiết hơn và đưa ra một solution trong Section 13.3.